

HelixCluster

HelixCluster

Дистрибуирани оперативни систем за AI рачунање — од GPU-ова у податковним центрима до ручних уређаја на ивици мреже, под јединственом контролном равни.

Сажетак

Helix Цлустер OS је оперативни систем следеће генерације који оркестрира рачунање на хетерогеним чворовима — од GPU-ова у податковним центрима до једноплочних рачунара на ивици мреже и ручних уређаја — обједињујући распоредивање НРС задатака, оркестрацију контејнера, AI/ML инференцију, федерисано управљање више кластера и сигурне сесије за више корисника под јединственом контролном равни.

Кратак опис

Дистрибуирани OS заснован на Go / кластер за дељење GPU ресурса. Обједињује распоредивање НРС задатака (двослојни распоредник по моделу Омега), оркестрацију контејнера, рутирање AI инференције, федерацију и сигурне сесије за више корисника на хетерогеним чворовима, координисано путем SWIM трача и Raft консензуса, са постквантним енкрипцијом од краја до краја.

Детаљан опис

Helix Цлустер OS оркестрира рачунарске задатке на радикално хетерогеном хардверу — GPU-овима у податковним центрима, једноплочним рачунарима на ивици мреже, па чак и ручним уређајима — под јединственом контролном равни, третирајући полицу A100 чипова и шаку једноплочних рачунара као један адресабилни систем уместо као десетак неповезаних острва. То је Go радни простор (монорепозиторијум са гит подмодулима) који имплементира

седмослојни стек, од L0 хардверског слоја до L7 федерације и опсервабилности, координисан четрнаест микросервиса контролне равни. Чланство чворова прати се путем SWIM трача и откривања, тако да се систем сам поправља како чворови приступају и напуштају мрежу; конзистентно стање осигурава Raft консензус, организован у Raft групе по сегментима са локалним читањем код власника закупа ради брзине и STONITH оградом како би се гарантовало да партиционисани чвор не може оштетити дељено стање. Распоредивање задатака пролази кроз двослојни распоредник по моделу Омега — оптимистична конкурентност, упаривање ClassAd-ова, групно распоредивање, преемптивно распоредивање на основу мултипликатора вредности и постављање засновано на ограничењима — а затим иде даље од класичног HPC распоредника: рутирање свесно угљеничног отиска и трошкова/ТСО-а, аутоматско скалирање према облаку у случају наглог оптерећења и адаптери за тржишта (Akash, ио.нет, RunPod, AWS Спот, Chutes) који омогућавају да се послови пребаце на изнајмљене ресурсе када локални капацитети буду исцрпљени.

Крајњи корисници не виде директно ниједан од тих механизма; они комуницирају путем чистог модела сесија (алокација рачунарских ресурса), интерактивног WebSocket/PTY терминала, интерне руте за AI инференцију и читавања искоришћености ресурса. Сигурност је интегрални слој, а не накнадни додатак: SPIFFE идентитет, атестација уређаја (изазов/одговор, доказ GPU рада, запечаћивање), контролна тачка за извоз (KYC), као и постквантни енкриптовани транспорт од краја до краја, заснован на хибридној размени кључева X25519 + ML-KEM-768 са AEAD заштитом записа и одбацивањем поновљених порука — пројектован тако да и данас ухваћен саобраћај остане поверљив чак и пред будућим квантним противником. Исправност се не проглашава, већ *доказује*: детерминистичко симулационо тестирање (FoundationDB-стил са унапред задатим извршавањима, убацивањем грешака, симулацијом мреже, бајт-по-бајт репродукцијом и Порцупине провером линеарности) репродукује дистрибуиране грешке по потреби, а обавезни упарени тестови мутација доказују да заштитни тестови заиста хватају грешке. Архитектура и документација одржавају се поузданим помоћу механичких провера које прекидају изградњу чим се стварност и документација разилазе.

Зашто смо га изградили

Да бисмо покретали AI и HPC послове на крајње различитим хардверским нивоима без спајања засебних распоредивача, оркестратора и стекова за инференцију — и то уз инжењерску гаранцију да свака испоручена функција доказује *стварно*

понашање крајњих корисника (никад зелени тестови уместо стубова) и да свака специфична могућност оперативног система користи стварни нативни алат по платформи (нема лажних Linux симулација). Мотивациони проблем, наведен у управљању репозиторијума, јесте режим грешке „тестови пролазе, али функција заправо не ради“, који је пројекат експлицитно изграђен да елиминише.

Зашто је револуционаран

Он спаја пет ствари које су обично пет одвојених стекова — НРС распоредивање, оркестрацију контејнера, AI инференцију, федерацију више кластера и сигурне сесије за више корисника — у јединствену контролну раван која се протеже од GPU-ова у податковним центрима све до ручних уређаја на ивици мреже. И то чини уз ригорозан буџет који се обично резервише за специјализовану инфраструктуру: коректност на нивоу формалних метода (TLA+ спецификације, детерминистичка симулација, провера линеарности) и поверљиви транспорт отпоран на квантне нападе, гаранције које већина оркестратора једноставно не покушава да оствари. Поред техничке диференцијације, распоредивање свесно трошкова и угљеничног отиска, као и могућност проширења путем тржишта у облаку, чине га и *економским* полугама — распоредивач аутоматски тражи јефтинији, еколошки прихватљивији или резервни капацитет, тако да исти посао кошта мање и производи мање емисија без потребе да ико преписује код.

Шта је иновативно

- **Детерминистичко тестирање симулацијом (DST)** — засејана, у потпуности репродуктивна симулација која убацује грешке, временске разлике и партиционисање мреже, понавља их бајт по бајт и проводи резултат кроз Порцупине проверавач линеарности, тако да се Хајзенбаг ухваћен једном може репродуковати по команди заувек.
- **Двонивовски распоредивач по Омеги-моделу** — оптимистичко распоредивање с конкурентношћу уз подударање ClassAd, групно распоредивање и преемптивно распоредивање с мултипликатором вредности, дизајн дељеног стања који омогућава да више распоредивача истовремено приступа једном кластеру без централног уског грла.
- **Пост-квантна E2EE / поверљива инференција** — хибридна размена кључева X25519 + ML-KEM-768 с везивањем пара кључева за одговор по захтеву и AEAD с одбацивањем поновљених порука (криптографски примитиви су стварни и тестирани; потпуна поверљива

размена између више чворова експлицитно је ПЛАНИРАНА/ограничена).

- **Доказно засновано поверење** — чворови морају да докажу шта јесу: SPIFFE идентитет, доказ о GPU раду, запечаћивање уређаја, контролна капија за извоз и генерисање документације за усклађеност са AI законом ЕУ, тако да се поверење заснива на доказима, а не претпоставља на основу положаја у мрежи.
- **Оркестрација свесна трошкова и угљеничног отиска** — моделирање укупних трошкова власништва, распоредивање свесно угљеничног отиска, проширење у облак, Н+К резервни капацитет и адаптери за тржишта у облаку, чинећи цену и емисије примарним улазним параметрима распоредивања уместо накнадних размишљања.
- **Консензус с више Raft група** — Raft групе по сегментима с локалним читањем код носиоца закупа за конзистентност ниске латенције, подржане STONITH оградом (IPMI / EC2 / Azure / SBD) тако да се заглављени чвор одлучно уклања, а не оставља да наруши стање.
- **Механичке провере против одступања** — archlint прекида изградњу чим документована компонента мапира на путању пакета која не постоји, а ланац документације одржава бајт-прецизну конзистентност између Markdowna, HTML, PDF и DOCX, тако да документација не може тихо да лаже о коду.

Највећи технички изазови и како смо их решили

- **“ПАСС-блуп” (тестови пролазе на нефункционалним карактеристикама).** Режим грешке који је цео пројекат дизајниран да елиминише: зелени сет тестова над шаблонима. Решено обавезним упареним мутационим тестирањем — свака радна ставка носи именовани тест-чувар који *мора да падне* под независну мутацију кода пре него што се ставка означи као завршеном, тако да пролазни тест доказиво проверава стварно понашање, а не само макету.
- **Паритет између платформи (нема макета само за Linux).** Решено заједничким интерфејсом подељеним помоћу буилд тагова на аутентичне OS-специфичне функционалности — Linux цгрупп / /proc / WireGuard језгра, macOS sysctl / vm_stat / IOKit / wireguard-go — а затим унакрсно проверено помоћу независног OS оракла, тако да свака платформа приказује стварно стање система уместо фикције засноване на Linuxu.
- **Дистрибуирана исправност у условима грешака.** Решено детерминистичким симулационим тестирањем и цхецкером линеарности који генерише и репродукује

партиционисање, падове система и временске разлике, подржано формалним спецификацијама TLA+ које прецизно дефинишу инваријанте консензуса и распоређивања пре него што се напише иједна линија кода.

- **Документација и одступање архитектуре.** Решено помоћу archlint-a, који прекида буилд ако постоји документовани али непостојећи мапирани пакет, и верификационог гејта docs_chain без могућности заобилажења — одступање је прекид буилда, а не застарела вики страница.
- **Поштено дефинисање обима недовршеног посла.** Повратна спрега за мултинодну инференцију с поверљивим подацима намерно је заштићена тикетом и означена као “joш није валидирана од краја до краја” уместо да се представља као испоручена — иста дисциплина примењена је на оно што *joш није урађено* као и на оно што јесте.

Технички стек

- **Go (го.мод: 1.25 / тоолсхаин 1.26.4)** — језик контролне равни у радном простору од ~30 модула; изабран због јефтине гороутине конкурентности и статичких бинарних фајлова који се идентично распоређују од податковног центра до ивице мреже.
- **Zig (0.14+) + Ц/С++** — коришћен тамо где Go рунтима представља препреку: за нисконивојске системске примитиве и GPU језгра којима је потребан детерминистички, алокацијски контролисан приступ хардверу.
- **gRPC + Protocol Buffers** — свака међусубсистемска API (api/v1/) је типизирани, верзионирани уговор, тако да четрнаест микросервиса еволуира без међусобног нарушавања или ручног креирања формата за пренос података.
- **Raft (етцд-рафт) + SWIM госсип** — намерна подела: Raft преноси стање које *мора* бити строго конзистентно, док SWIM госсип управља чланством и откривањем на великој скали где би консензус био превише захтеван.
- **PostgreSQL 16, Redis 7 кластер, етцд в3.5, SQLite** — право складиште за сваки задатак: PostgreSQL за трајно релационо стање, Redis за врући кеш, етцд за координацију, а уграђени SQLite за локални регистар радних ставки НХС чвора.
- **NATS 2.10 (JetStream), Kafka 4.0 (KRaft), RabbitMQ 3.13** — три окоснице за поруке за три врсте саобраћаја: NATS/JetStream за брзо интерно слање догађаја, Kafka за трајна струјања података високог протока, RabbitMQ за класичну сематику брокера.
- **WireGuard мрежа + ML-KEM-768/X25519 + AES-256-GCM/ChaCha20-Poly1305 + HKDF** — WireGuard за

мршаву мрежу чвор-чвор, умотан у хибридни постквантни хандсхаке и AEAD записе како би транспорт био поверљив и пред класичним и пред квантним нападачима.

- **SPIFFE + JWT (HS256) + RBAC заснован на опсезима + ОРА** — слојевита идентификација и ауторизација: SPIFFE за идентитет радног оптерећења, JWT за токене, RBAC заснован на опсезима за грубу контролу приступа, а ОРА за изражавање фино гранулиране политике као кода.
- **Prometheus v2.50, Grafana 10.4, Jaeger 1.55, W3C праћење** — метричке вредности, контролне табле и дистрибуирано праћење с пропагацијом W3C контекста, тако да се захтев може пратити кроз сервисе и хардверске слојеве.
- **HashiCorp Vault 1.16** — тајне и криптографски материјал држе се изван кода и конфигурације и издају под ревизијом.
- **Docker Цомпосе, Kubernetes (кустомизе, ојачани securityContext), Helm** — Цомпосе за локално покретање, а Kubernetes/Helm с ојачаним securityContext-ом за стварне имплементације, с једном дефиницијом која се промовише кроз различита окружења.
- **React + TypeScript + Vite (Node 20+)** — брзо, типизирано веб сучеље за сесије, терминале и искоришћеност ресурса.
- **TLA+** — формална спецификација инваријаната консензуса и распоређивања, тако да се најтеже тестабилна својства доказују на нивоу дизајна пре саме имплементације.

Напомене о статусу и искрености

- **Статус: у развоју.** Верзија је рана (0.1.0-dev). Неколико напредних функција — потпуно поверљива мулти-чворовна инференција са повратном спрегом, обрачун на тржишту и попуњавање распореда вођено потврдама — експлицитно су означене као ЗАМИШЉЕНЕ / инфраструктурно ограничене у репозиторијуму и **нису** представљене као потпуно функционалне. Покривеност је самопријављена.
- **Лиценца: није одређена.** Није јасно наведена; дијаграм Helm садржи непроверене привремене ознаке HelixCluster/HelixCluster и URL-ове helixcluster.io који се не поклапају са стварним удаљеним ресурсима.
- Пројекти у склопу LLM стека (LLMOrchestrator, LLMProvider, LLMsVerifier) јесу одвојени потмодули, а не модел сервери који се хостују унутар кластера.

Приоритетни ниво: Helix-пример (LLM-инфраструктурни кластер — рачунарска основа која може да хостује задатке инференције и рачунања). Рангира се иза HelixTrack.